LABSHEET-02

1.Create a file named notes.txt and display its permissions string using the ls -l command.

touch notes.txt

ls -l notes.txt

2. Change the permissions of notes.txt to 644 using numeric (octal) mode, and verify the changes using ls -l.

chmod 644 notes.txt

ls -l notes.tx

3. Change notes.txt permissions to 600 (owner read/write only, no permissions for group and others) and verify.

chmod 600 notes.txt

ls -l notes.txt

4. Create an executable file named script.sh and grant execute permission exclusively to the owner using symbolic mode.

touch script.sh

chmod u+x,go-x script.sh

ls -l script.sh

chmod 700 script.sh

5. Use the whoami command and ls -l notes.txt to identify the user and group ownership of notes.txt.

whoami

ls -l notes.txt

6. Run the ps command in your terminal and record how many processes are active in your current shell session.

ps

7. Run ps aux and count or estimate the total number of running processes across the entire operating system.

ps aux

ps aux | wc -l

ps -e --no-headers | wc -l

8. Start a background process using sleep 60 & and immediately run jobs to verify that it is active in the background.

sleep 60 &

jobs

9. Use the kill command with the Process ID (PID) to terminate the active sleep process created in Exercise

```
sleep 60 &

pid=$!

kill "$pid"
```

10. Search for an available software package named tree using your system's package manager without installing it.

```
apt search tree

apt-cache policy tree

dnf search tree

dnf info tree
```

11. Display the current umask value of your terminal shell environment.

```
umask
```

12. Use symbolic chmod syntax to remove write permissions from the group on a file named document.txt.

```
chmod g-w document.txt

ls -l document.txt
```

13. Bring a background job with Job ID 1 back to the foreground using the fg command.

```
fg %1
```

14. Send a background job with Job ID 1 to resume execution in the background using the bs command.

```
bg %1
```

15. Display the kernel information and system architecture using uname -a.

```
uname -a
```

16. Display a snapshot of all processes running for your user account using ps -u $USER.

```
ps -u "$USER"
```

17. Use pkill to terminate a test process running under the name sleep.

```
pgrep -a -x sleep

pkill -x sleep
```

18. Check the current repository package index updates on an Ubuntu/Debian system using sudo apt update.

```
sudo apt update
```

19. Display the list of currently installed packages on your system using apt list --installed (or dnf list installed).

```
apt list --installed
```

Fedora:

dnf list installed

20. Use chown to change the owner of script.sh to yourself explicitly, and verify with ls -l.

sudo chown "$USER" script.sh

ls -l script.sh

sudo chown "$USER":"$(id -gn)" script.sh

21. Set permissions on script.sh so that the owner has full access (rwx), the group has read and execute (rx), and others have zero permissions — write the numeric command.

chmod 750 script.sh

ls -l script.sh

22. Explain the exact difference in rights between running chmod 700 file and running chmod 007 file.

23. Use chown to change both the user owner and group owner of a file to student:student in a single command.

sudo chown student:student file

ls -l file

24. Launch the top interactive process monitor, press 'P' to sort processes by CPU utilization, and press 'q' to exit.

top

Inside top:

1. Press **P** to sort by CPU utilization.
2. Press **q** to quit.

25. Start two background sleep processes (sleep 100 & and sleep 200 &), list them with jobs, and bring job 1 to the foreground using fg.

sleep 100 &

sleep 200 &

jobs -l

fg %1

26. Use pkill with a safe target process name to terminate all active instances of that command simultaneously.

pgrep -a -x sleep

pkill -x sleep

27. Install the tree package using your system's package manager (apt or dnf), and run tree on your home directory.

sudo apt update

sudo apt install tree

tree ~

Fedora:

sudo dnf install tree

tree ~

28. Remove the tree package atier testing, and verify its uninstallation using the package manager list command.

sudo apt remove tree

apt list --installed tree

sudo dnf remove tree

dnf list installed tree

29. Check your default umask setting, create a new file named test_mask.txt, and analyze how umask influenced its permissions.

umask

touch test_mask.txt

ls -l test_mask.txt

30. Filter system processes to list only those belonging to your logged-in username using ps aux | grep $USER.

ps aux | grep "$USER"

ps -u "$USER"

31. Change group ownership of a directory named project_dir to group devteam using the chgrp command with sudo.

sudo chgrp devteam project_dir

ls -ld project_dir

32. Launch a process with a lower scheduling priority using nice -n 10 sleep 300 & and confirm its priority using ps or top.

nice -n 10 sleep 300 &

jobs -l

ps -o pid,ni,pri,stat,cmd -p <PID>

33. Modify the priority of an already running process using the renice command by specifying its PID.

renice 10 -p <PID>

ps -o pid,ni,cmd -p <PID>

34. Search for the apache2 or nginx package in apt/dnf, install it, and check its system status using systemctl status.

sudo apt update

sudo apt install apache2

systemctl status apache2

35. Explain why executing apt install <package> without running apt update first on a fresh session might cause installation failure.

sudo apt update

sudo apt install package-name

36. Create a shell script file, grant execute permissions using symbolic mode (chmod +x), execute it with ./script.sh, and explain why execution fails if the shebang line (#!/bin/bash) is missing or invalid.

cat > script.sh <<'EOF'

#!/bin/bash

echo "Hello from Linux"

EOF

chmod +x script.sh

./script.sh

37. Set up permissions for shared.txt so the owner has read/write, the group has read-only, and others have zero access — write out both symbolic AND numeric commands.

chmod u=rw,g=r,o= shared.txt

chmod 640 shared.txt

ls -l shared.txt

38. Start a background task with nice -n 15 sleep 300 &, locate its PID in top, and explain how the 'nice value' impacts CPU time allocation.

nice -n 15 sleep 300 &

jobs -l

ps -o pid,ni,pri,%cpu,cmd -p <PID>

39. Debug Scenario: A student runs kill 1234 but the process refuses to terminate. What escalating command should be used next, and why is it considered a 'last resort'?

ps -p 1234 -o pid,ppid,stat,cmd

kill -TERM 1234

kill -KILL 1234

40. Display the top 5 CPU-consuming processes currently active on the system using ps aux --sort=-%cpu | head -5

ps aux --sort=-%cpu | head -n 6

41. Construct a single-line command using find and chmod to grant execute permissions (+x) to all .sh files in your working directory tree simultaneously.

find . -type f -name "*.sh" -exec chmod +x {} +

42. Explain the security risks associated with running chmod 777 on a sensitive system configuration file in production.

chmod 777 sensitive.conf

43. Install a package (e.g., tree), and use the low-level package tool (dpkg -L <pkg> or rpm -ql <pkg>) to list every file installed onto the filesystem by that package.

dpkg -L tree

rpm -ql tree

44. Conceptual Analysis: Explain the difference between process termination via SIGTERM (Signal 15) vs SIGKILL (Signal 9), and why applications should handle SIGTERM.

kill -TERM PID

kill -KILL PID

45. Create two files owned by different users (using sudo chown), then write a single find command that filters and lists files owned by a specific user.

sudo touch user1_file.txt user2_file.txt

sudo chown user1:user1 user1_file.txt

sudo chown user2:user2 user2_file.txt

find . -type f -user user1

46. Set the Set User ID (SUID) bit on an executable binary custom_tool, write the chmod command, and explain its execution privilege implications.

sudo chmod u+s custom_tool

ls -l custom_tool

sudo chmod 4755 custom_tool

47. Set the Set Group ID (SGID) bit on a shared folder /shared_dir using octal mode (2775), and explain how newly created files inherit group ownership.

sudo chmod 2775 /shared_dir

ls -ld /shared_dir

drwxrwsr-x

48. Set the Sticky Bit on a directory /public_tmp using symbolic (chmod o+t) or octal (chmod 1777) mode, and explain why it prevents unauthorized file dele□on.

sudo chmod o+t /public_tmp

sudo chmod 1777 /public_tmp

drwxrwxrwt

49. Demonstrate job management: Suspend a running command using Ctrl+Z, verify its stopped state with jobs, resume it in the background with bg, and then attach it with fg.

sleep 300

Jobs

bg

fg

50. Write a shell command pipeline that identifies the process consuming the most memory (%MEM), logs its PID and details to a file named top_memory.log, and gracefully requests its termination using SIGTERM.

ps aux –sort=-%men / head -n 2 >

top_memory.log.

cat top_memory.log

PID=$(ps aux –sort=-%men / awk

‘NR==2 {print $2} ‘)

Kill -TERM “$PID”